

私人保管箱：運用 AlloyDB 資料列層級安全性機制，打造「零信任智慧」

來源：<https://codelabs.developers.google.com/zero-trust-agents-with-alloydb?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將建構「私人保險箱」：安全的人資助理，可根據登入者動態變更回覆內容。您將瞭解如何在 AlloyDB 中使用 PostgreSQL 資料列層級安全防護機制 (RLS)，保護 AI 應用程式。

目錄

1. [1. 總覽](#)
2. [2. 事前準備](#)
3. [3. 資料庫設定](#)
4. [4. 結構定義佈建](#)
5. [5. 啟用及強制執行安全性](#)
6. [6. 建立存取權政策](#)
7. [7. 建構「Chameleon」應用程式](#)
8. [8. 驗證零信任](#)
9. [9. 清除所用資源](#)
10. [10. 恭喜](#)

1. 總覽

在急於建構生成式 AI 應用程式時，我們經常會忘記最重要的元件：**安全性**。

假設您要建構人力資源聊天機器人，您希望 AI 能夠回答「我的薪水是多少？」或「我的團隊表現如何？」等問題。

- 如果艾莉絲 (一般員工) 提出要求，她應該只會看到自己的資料。
- 如果Bob (經理) 提出要求，他應該會看到自己團隊的資料。

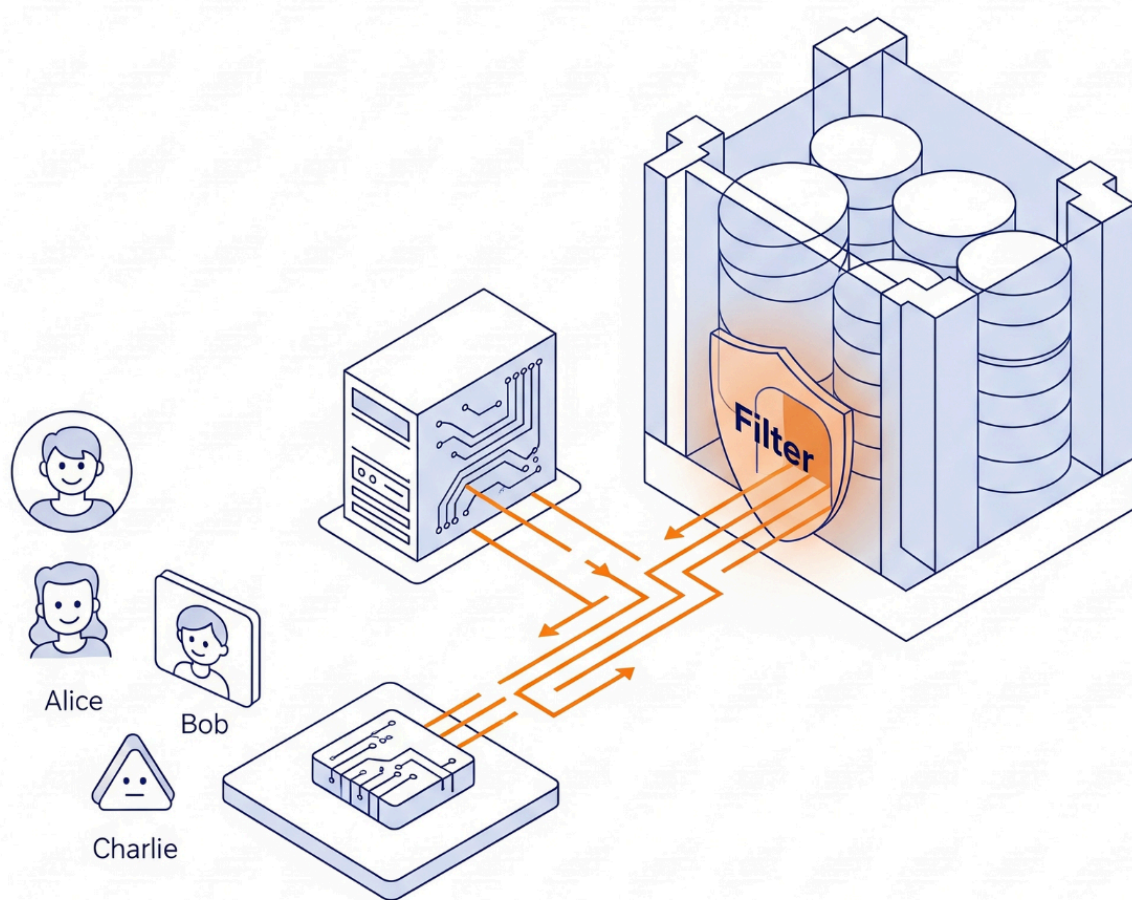
問題

大多數 RAG (檢索增強生成) 架構都會嘗試在應用程式層中處理這項問題。他們會在擷取區塊後進行篩選，或是仰賴 LLM 「表現良好」。這很脆弱。如果應用程式邏輯失敗，就會發生資料外洩。

解決方法

將安全性下推至資料庫層。在 AlloyDB 中使用 PostgreSQL 資料列層級安全性 (RLS)，可確保資料庫實際拒絕傳回使用者無權查看的資料，無論 AI 要求什麼。

在本指南中，我們將建構「私人保險箱」：這是一個安全的人資助理，會根據登入者身分動態變更回覆內容。



架構

我們不會在 Python 中建構複雜的權限邏輯，我們使用的是資料庫引擎本身。

1. 介面：模擬登入的簡單 Streamlit 應用程式。
2. 大腦：AlloyDB AI (與 PostgreSQL 相容)。

3. **運作機制**：我們會在每筆交易開始時設定工作階段變數 (`app.active_user`)。資料庫政策會自動檢查 `user_roles` 資料表 (做為身分識別提供者)，以篩選資料列。

建構項目

安全的人資助理應用程式。您不必依賴應用程式邏輯來篩選機密資料，而是直接在 AlloyDB 資料庫引擎中實作資料列層級安全性 (RLS)。這樣一來，即使 AI 模型「產生幻覺」或嘗試存取未經授權的資料，資料庫也會實際拒絕傳回資料。

課程內容

您將學會：

- 如何設計 RLS 的結構定義 (區分資料與身分)。
- 如何編寫 PostgreSQL 政策 (`CREATE POLICY`)。
- 如何使用 `FORCE ROW LEVEL SECURITY` 略過「資料表擁有者」豁免。
- 如何建構 Python 應用程式，為使用者執行「情境切換」。

需求條件

- 瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
 - 已啟用計費功能的 Google Cloud 專案。
 - 存取 Cloud Shell 或已安裝 gcloud 和 psql 的終端機。
-

2. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 啟用必要的 API：按照[這個連結](#)啟用 API。

或者，您也可以使用 gcloud 指令執行這項操作。如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

```
gcloud services enable \  
  alloydb.googleapis.com \  
  compute.googleapis.com \  
  cloudresourcemanager.googleapis.com \  
  servicenetworking.googleapis.com \  
  aiplatform.googleapis.com
```

常見錯誤與疑難排解

「幽靈專案」症候群	您執行了 <code>gcloud config set project</code> ，但實際上在控制台 UI 中查看的是其他專案。檢查左上方的下拉式選單中的專案 ID！
帳單 路障	您已啟用專案，但忘記帳單帳戶。AlloyDB 是高效能引擎，如果「油箱」(帳單) 空了，就無法啟動。
API 傳播 延遲	您點選了「啟用 API」，但指令列仍顯示 <code>Service Not Enabled</code> 。等待 60 秒。雲端需要一點時間喚醒神經元。
配額 Quags	如果您使用全新的試用帳戶，可能會達到 AlloyDB 執行個體的區域配額。如果 <code>us-central1</code> 失敗，請嘗試 <code>us-east1</code> 。

3. 資料庫設定

在本實驗室中，我們會使用 AlloyDB 做為測試資料的資料庫。並使用「叢集」保存所有資源，例如資料庫和記錄檔。每個叢集都有一個「主要執行個體」，可做為資料的存取點。資料表會保存實際資料。

我們來建立 AlloyDB 叢集、執行個體和資料表，以便載入測試資料集。

1. 按一下按鈕，或將下方連結複製到已登入 Google Cloud 控制台使用者的瀏覽器。
2. 完成這個步驟後，存放區會複製到本機 Cloud Shell 編輯器，您就能從專案資料夾執行下列指令 (請務必確認您位於專案目錄中)：

```
sh run.sh
```

3. 現在請使用 UI (按一下終端機中的連結，或按一下終端機中的「preview on web」連結)。
4. 輸入專案 ID、叢集和執行個體名稱的詳細資料，即可開始使用。
5. 在記錄捲動時去買杯咖啡吧！如要瞭解系統在幕後如何執行這項操作，請參閱這篇文章。這個過程大約需要 10 到 15 分鐘。

重要事項：如要在本機或從任何位置進行測試，請前往 AlloyDB 執行個體，按一下「EDIT」，然後按一下「Enable Public IP」或「Public IP Connectivity」(不是輸出 IP)，並在「Authorized External Networks」中輸入「0.0.0.0/0」，以供開發用途。完成後，請移除並停用公開 IP 連線。

常見錯誤與疑難排解

「耐心」問題	資料庫叢集是龐大的基礎架構，如果因為「看起來卡住」而重新整理頁面或終止 Cloud Shell 工作階段，您可能會得到「虛擬」執行個體，這類執行個體已部分佈建，但必須手動介入才能刪除。
區域不符	如果您在 us-central1 中啟用 API，但嘗試在 asia-south1 中佈建叢集，可能會遇到配額問題或服務帳戶權限延遲。整個實驗室都使用同一個區域！
殭屍叢集	如果您先前使用過相同名稱的叢集，但未刪除，指令碼可能會顯示叢集名稱已存在。叢集名稱在專案內不得重複。
Cloud Shell 超時	如果咖啡休息時間為 30 分鐘，Cloud Shell 可能會進入休眠狀態，並中斷 sh run.sh 程序。請保持分頁啟用狀態！

步驟 4 · 約 0 分鐘

4. 結構定義佈建

本步驟將涵蓋下列內容：

Step 1: Connect & Prepare

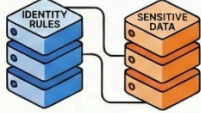


Authenticate to AlloyDB Studio
Sign in using your "postgres" user and the password you created for the cluster.



Open the SQL Editor
All commands will be entered and executed within an editor window.

Step 2: Create the Schema



Use a Two-Table Structure
Separate identity rules from sensitive data to avoid "Infinite Recursion" errors.

username	role	
Alice	employee	
Bob	manager	
Charlie	employee	

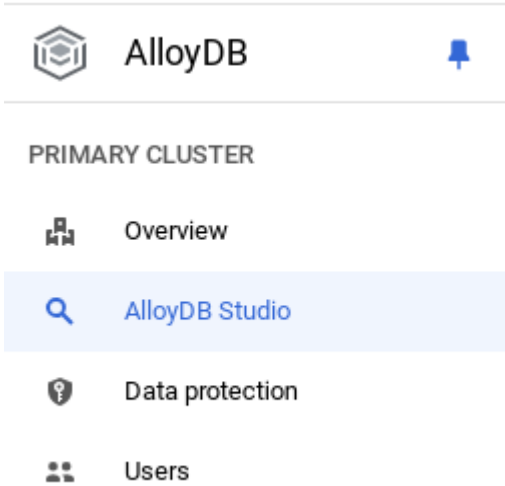
Create the Identity Table ('user_roles')
This table defines user permissions and acts as the identity provider.

name	salary	performance_review
Alice	80000	Meets expectations...
Bob	120000	Strong leader...
Charlie	85000	Exceeds expectations...

Create the Data Table ('employees')
This table stores the sensitive employee information that requires access control.

詳細步驟如下：

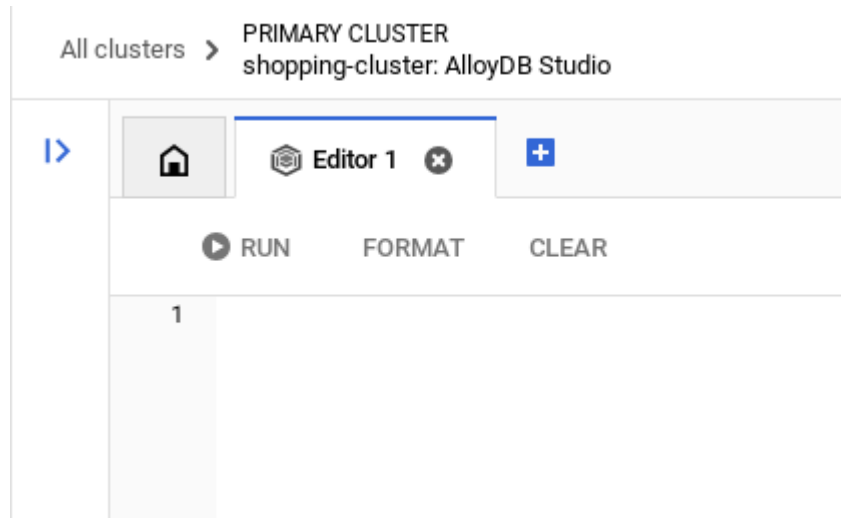
AlloyDB 叢集和執行個體啟動後，請前往 AlloyDB Studio SQL 編輯器啟用 AI 擴充功能，並佈建結構定義。



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」(或您在建立時設定的密碼)

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

建立資料表

我們需要兩個資料表：一個用於機密資料 (員工)，另一個用於身分識別規則 (user_roles)。請務必將兩者分開，以免政策發生「無限遞迴」錯誤。

您可以在 AlloyDB Studio 中使用下列 DDL 陳述式建立資料表：

```
-- 1. Create User Roles (The Identity Provider)
CREATE TABLE user_roles (
    username TEXT PRIMARY KEY,
    role TEXT -- 'employee', 'manager', 'admin'
);

INSERT INTO user_roles (username, role) VALUES
('Alice', 'employee'),
('Bob', 'manager'),
('Charlie', 'employee');

-- 2. Create the Data Table
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name TEXT,
    salary INTEGER,
    performance_review TEXT
);

INSERT INTO employees (name, salary, performance_review) VALUES
('Alice', 80000, 'Alice meets expectations but needs to improve punctuality.'),
('Bob', 120000, 'Bob is a strong leader. Team morale is high.'),
('Charlie', 85000, 'Charlie exceeds expectations. Ready for promotion.');
```

常見錯誤與疑難排解

在員工資料表內定義角色時，系統偵測到

失敗原因：如果政策內容為「請檢查員工資料表，確認我是否為管理員」，資料庫必須查詢資料表來檢查政策，這會再次觸發政策。結果：偵測到無限遞迴。**修正方式：**請一律保留個別的對照表

無限遞迴

(user_roles)，或使用實際的資料庫使用者來擔任角色。

驗證資料：

```
SELECT count(*) FROM employees;  
-- Output: 3
```

5. 啟用及強制執行安全性

現在開啟防護盾，我們也會建立 Python 程式碼用來連線的通用「應用程式使用者」。

從 AlloyDB 查詢編輯器執行下列 SQL 陳述式：

```
-- 1. Activate RLS
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

-- 2. CRITICAL: Force RLS for Table Owners
ALTER TABLE employees FORCE ROW LEVEL SECURITY;

-- 3. Create the Application User
DO
$do$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname = 'app_user') THEN
        CREATE ROLE app_user LOGIN PASSWORD 'password';
    END IF;
END
$do$;

-- 4. Grant Access
GRANT SELECT ON employees TO app_user;
GRANT SELECT ON user_roles TO app_user;
```

常見錯誤與疑難排解

以 postgres (超級使用者) 進行測試，並查看所有資料。	失敗原因： 根據預設，RLS 不適用於資料表擁有者或超級使用者。這會略過所有政策。 疑難排解： 如果政策似乎「失效」(允許所有內容)，請檢查您是否以 <code>postgres</code> 身分登入。 修正： <code>FORCE ROW LEVEL SECURITY</code> 指令可確保即使是擁有者，也必須遵守規則。這對測試至關重要。
----------------------------------	---

6. 建立存取權政策

我們將使用稍後從應用程式程式碼設定的工作階段變數 (app.active_user)，定義兩項規則。

從 AlloyDB 查詢編輯器執行下列 SQL 陳述式：

```
-- Policy 1: Self-View
-- Users can see rows where their name matches the session variable
CREATE POLICY "view_own_data" ON employees
FOR SELECT
USING (name = current_setting('app.active_user', true));

-- Policy 2: Manager-View
-- Managers can see ALL rows.
CREATE POLICY "manager_view_all" ON employees
FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM user_roles
        WHERE username = current_setting('app.active_user', true)
        AND role = 'manager'
    )
);
```

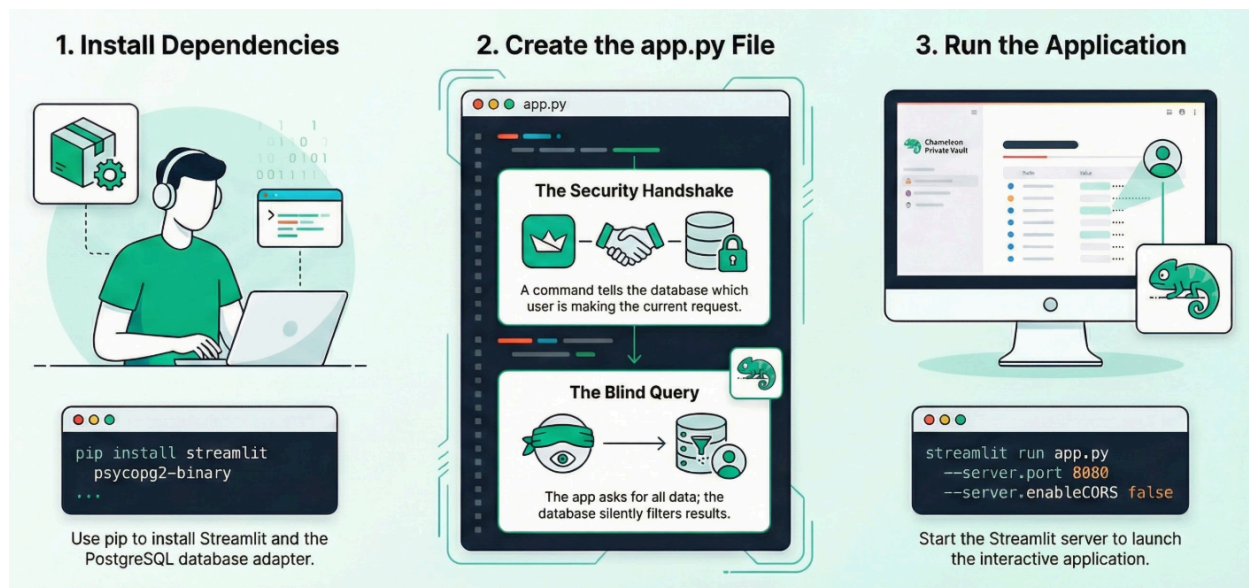
常見錯誤與疑難排解

使用 current_user 而非 app.active_user。	問題：Current_user 是保留的 SQL 關鍵字，會傳回資料庫角色 (例如 app_user)。我們需要應用程式使用者 (例如 Alice)。修正：請一律使用自訂命名空間，例如 app.variable_name。
忘記在 current_setting 中加入 true 參數。	問題：如果未設定變數，查詢會因錯誤而當機。修正方式：current_setting('...', true) 會傳回 NULL，而非當機，因此安全地傳回 0 個資料列。

步驟 7 · 約 0 分鐘

7. 建構「Chameleon」應用程式

我們將使用 Python 和 Streamlit 模擬應用程式邏輯。



在編輯器模式中開啟 Cloud Shell 終端機，然後前往根資料夾或要建立這個應用程式的目錄。建立新資料夾。

1. 安裝依附元件：

在 Cloud Shell 終端機中從新專案目錄內執行下列指令：

```
pip install streamlit psycopg2-binary
```

2. 建立 app.py：

建立名為 **app.py** 的新檔案，然後複製[存放區檔案](#)的內容。

```

import streamlit as st
import psycopg2

# CONFIGURATION (Replace with your IP)
DB_HOST = "10.x.x.x"
DB_NAME = "postgres"
DB_USER = "postgres"
DB_PASS = "alloydb"

def get_db_connection():
    return psycopg2.connect(
        host=DB_HOST, database=DB_NAME, user=DB_USER, password=DB_PASS
    )

def query_database(user_name):
    conn = get_db_connection()
    try:
        with conn.cursor() as cur:
            # THE SECURITY HANDSHAKE
            # We tell the database: "For this transaction, I am acting as..."
            cur.execute(f"SET app.active_user = '{user_name}';")

            # THE BLIND QUERY
            # We ask for EVERYTHING. The database silently filters it.
            cur.execute("SELECT name, role, salary, performance_review FROM employees;")
            return cur.fetchall()
    finally:
        conn.close()

# UI
st.title("🔒 The Private Vault")
user = st.sidebar.radio("Act as User:", ["Alice", "Bob", "Charlie", "Eve"])

if st.button("Access Data"):
    results = query_database(user)
    if not results:
        st.error("🚫 Access Denied.")
    else:
        st.success(f"Viewing data as {user}")
        for row in results:
            st.write(row)

```

3. 執行應用程式：

在 Cloud Shell 終端機中從新專案目錄內執行下列指令：

```
streamlit run app.py --server.port 8080 --server.enableCORS false
```

常見錯誤與疑難排解

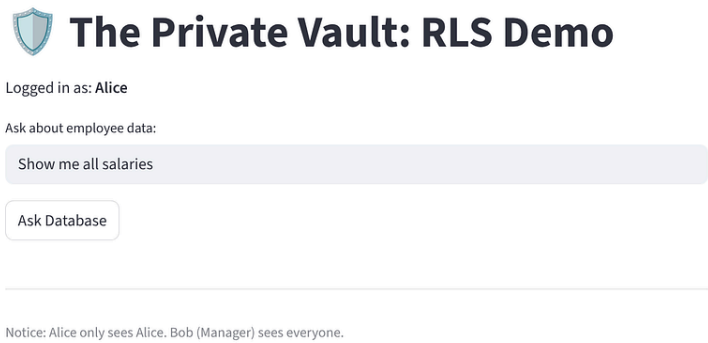
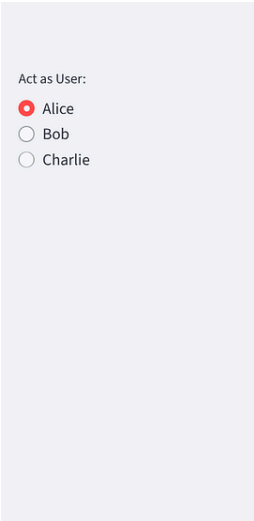
連線集區。	風險： 如果您使用連線集區，工作階段變數 SET app.active_user 可能會保留在連線上，並「洩漏」給下一個取得該連線的使用者。 修正方式： 在正式環境中，將連線傳回集區時，請一律使用 RESET app.active_user 或 DISCARD ALL。
Cloud Shell 顯示空白畫面。	修正方式： 使用通訊埠 8080 的「網頁預覽」按鈕。請勿點選終端機中的 localhost 連結。

步驟 8 · 約 0 分鐘

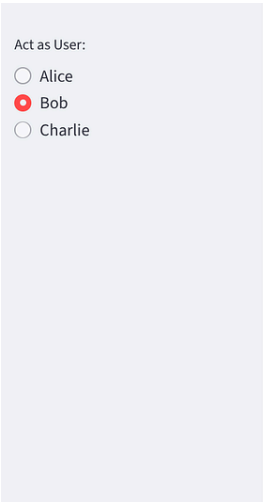
8. 驗證零信任

試用應用程式，確保零信任機制已導入：

選取「Alice」：她應該會看到 1 列 (自己)。



選取「Bob」：他應該會看到 3 列 (所有人)。



這對 AI 代理程式有何重要性

假設您要將模型連結至這個資料庫。如果使用者詢問模型「請總結所有績效評估」，模型會產生 `SELECT performance_review FROM employees`。

- 1. 沒有 RLS：模型會擷取所有人的私人評論，並洩漏給 Alice。
- 2. 使用 RLS：模型會執行完全相同的查詢，但資料庫只會傳回 Alice 的評論。

這就是零信任 AI。您不信任模型會篩選資料，因此強制資料庫隱藏資料。

將此項目移至正式版

這裡展示的架構屬於正式版，但為了方便學習，具體實作方式已簡化。如要在實際的企業環境中安全部署這項功能，請實作下列強化措施：

- 1. **實際驗證**：將「身分切換器」下拉式選單換成強大的身分識別提供者 (IDP)，例如 Google Identity Platform、Okta 或 Auth0。應用程式應先驗證使用者的權杖並安全地擷取身分，再設定資料庫工作階段變數，確保使用者無法偽造身分。

2. **連線集區安全**：使用連線集區時，如果處理方式不正確，工作階段變數有時可能會在不同使用者要求之間持續存在。請確保應用程式在將連線傳回集區時，會重設工作階段變數 (例如 RESET app.active_user) 或清除連線狀態，以免使用者之間發生資料外洩。
 3. **密鑰管理**：資料庫憑證硬式編碼會造成安全風險。使用 Google Secret Manager 等專用密鑰管理服務，在執行階段安全地儲存及擷取資料庫密碼和連線字串。
-

步驟 9 · 約 0 分鐘

9. 清除所用資源

完成本實驗室後，別忘了刪除 AlloyDB 叢集和執行個體。

前往 <https://console.cloud.google.com/alloydb/clusters>。按一下要刪除的叢集旁的垂直省略號，然後按一下「刪除」。

這項作業應會清理叢集及其執行個體。

步驟 10 · 約 0 分鐘

10. 恭喜

恭喜！您已成功將安全性下推至資料層。即使 Python 程式碼有嘗試 `print(all_salaries)` 的錯誤，資料庫也不會傳回任何內容給 Alice。

後續步驟

- 請使用自己的資料集試試看。
 - 請參閱 [AlloyDB AI 說明文件](#)。
 - 如要參加更多工作坊，請前往 [Code Vipassana](#) 網站。
-